

Solution Requirements

Date	30 September 2026
Team ID	PER206AIFITTRACK
Project Name	AI-FitTrack-API
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (What the system should do).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system (How the system should be).

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Med/Low)
1	User Authentication	POST /api/auth/register and /login with bcryptjs (10 rounds) hashing and JWT issuance; GET /profile with Bearer guard.	High
2	Workout CRUD Operations	Full CRUD endpoints: POST /workouts to create, GET /workouts to list, GET /:id, PUT /:id, DELETE /:id with user ownership.	High
3	Generative AI Integration	POST /api/ai/workout-recommendation integrates Google Gemini 1.5 Flash to generate tailored exercise routines.	High
4	AI Trend Analytics	POST /api/ai/fitness-insights evaluates logged workout volume, calorie burn, and consistency streaks with recovery advice.	High
5	Multi-Param Search & Filter	Query parameters on GET /workouts allowing filtering by workout type (Cardio, Strength, HIIT), date range, and duration.	Medium
6	Validation & Error Middleware	Mongoose schema validation and centralized Express error-handling middleware preventing unhandled crashes.	High
7	Data Persistence & Indexing	MongoDB collections for Users and Workouts with compound indexing on userId and date for sub-50ms query speeds.	High

Step 2: Non-Functional Requirements (NFR)

S.No	Requirement Category	Requirement Description	Target Metric / Benchmark
1	Performance & Speed	Asynchronous Node.js event-driven runtime; compound MongoDB indexing on userId and date.	CRUD response < 50ms; AI inference < 2.0s
2	Scalability	Stateless REST architecture allowing horizontal clustering via PM2 or Docker containers.	Supports 500+ requests/sec across workers
3	Security & Data Privacy	One-way bcrypt password hashing; HMAC-SHA256 signed JWT tokens; sensitive keys in .env.	Zero plain-text passwords; JWT expiry 30d
4	Reliability & Availability	Centralized error-handling middleware and resilient MongoDB reconnection logic.	99.9% uptime; zero unhandled promise rejections
5	Usability & Standards	Standard RESTful HTTP status codes (200, 201, 400, 401, 404, 500) and uniform JSON response envelopes.	100% compliant with OpenAPI REST conventions